

SUBMITTED BY: HARSHPREET KAUR (harshpreet.kaur@kenexai.com)

DATE OF SUBMISSION: 5 JUNE, 2026

› Question 1: Income Tax Calculator (Basic Operations)

↳ 2 cells hidden

› Question 2: Student Final Grade Calculator (Global & Local Variables)

↳ 2 cells hidden

› Question 3: String Slicing Operations

↳ 2 cells hidden

› Question 4: Collection Types - Inventory Management System

↳ 2 cells hidden

› Question 5 Grade Classification System (Conditions)

↳ 2 cells hidden

› Question 6 Password Validation System (Iterations)

↳ 2 cells hidden

Question 7: Data Processing with Loops (ETL Simulation)

↳ 2 cells hidden

Question 8: Pattern Matching - Data Format Processor

↳ 2 cells hidden

Question 9: String Manipulation - Text Analyzer

Description

Create a text analyzer that performs various string operations and uses collection types.

Requirements

1. Accept a paragraph of text as input
2. Perform the following operations:
 - Count total characters (including spaces)
 - Count words
 - Count sentences
 - Find most frequent word
 - Extract all unique words (using set)
 - Create a dictionary with word frequencies
 - Reverse the entire text
 - Extract first and last sentence

Input/Output Example

Enter text: Python is great. Python is easy. Python is powerful.

=== Text Analysis Report ===

Total Characters: 47

Total Words: 9

Total Sentences: 3

Most Frequent Word: Python (appears 3 times)

Unique Words: {'Python', 'is', 'great', 'easy', 'powerful'}

Word Frequencies: {'Python': 3, 'is': 3, 'great': 1, 'easy': 1, 'powerful': 1}

Reversed Text: .lufrewop si nohtyP .ysae si nohtyP .taerg si nohtyP

First Sentence: Python is great.

Last Sentence: Python is powerful.

Constraints

- Handle punctuation properly
- Case-insensitive word counting
- Use appropriate collection types

```
text = input("Enter text: ").strip()
if not text:
    print("Error: Text cannot be empty.")
else:
    total_characters = len(text)
    sentences = [s.strip() for s in text.split('.') if s.strip()]
    total_sentences = len(sentences)

    cleaned_text = text

    for char in ". , ! ? ; : " :
        cleaned_text = cleaned_text.replace(char, "")
    words = cleaned_text.lower().split()
    total_words = len(words)

    word_frequencies = {}

    for word in words:
        if word in word_frequencies:
            word_frequencies[word] += 1
        else:
            word_frequencies[word] = 1

    most_frequent_word = max(
        word_frequencies,
        key=word_frequencies.get
    )
    unique_words = set(words)
    reversed_text = text[::-1]

    first_sentence = sentences[0] + "." if sentences else ""
    last_sentence = sentences[-1] + "." if sentences else ""

    print("\n=== Text Analysis Report ===")
    print("Total Characters:", total_characters)
```

```

print("Total Words:", total_words)
print("Total Sentences:", total_sentences)

print(
    f"Most Frequent Word: {most_frequent_word} "
    f"(appears {word_frequencies[most_frequent_word]} times)"
)

print("Unique Words:", unique_words)

print("Word Frequencies:", word_frequencies)

print("Reversed Text:", reversed_text)

print("First Sentence:", first_sentence)

print("Last Sentence:", last_sentence)

```

Enter text: Python is great. Python is easy. Python is powerful.

=== Text Analysis Report ===

Total Characters: 52

Total Words: 9

Total Sentences: 3

Most Frequent Word: python (appears 3 times)

Unique Words: {'easy', 'great', 'is', 'python', 'powerful'}

Word Frequencies: {'python': 3, 'is': 3, 'great': 1, 'easy': 1, 'powerful': 1}

Reversed Text: .lufrewop si nohtyP .ysae si nohtyP .taerg si nohtyP

First Sentence: Python is great.

Last Sentence: Python is powerful.

Question 10: Comprehensive Calculator with All Concepts

Description

Create a comprehensive calculator that combines all concepts from Topics 1-4.

Requirements

1. Create a menu-driven calculator with the following operations:

- Basic arithmetic (+, -, , /, //, %, *)
- Comparison operations (compare two numbers)
- Logical operations (check conditions)
- String operations (concatenate, repeat)
- Collection operations (work with lists)

2. Use functions for each operation type
3. Use loops to keep the menu running until user exits
4. Use conditions to validate inputs and operations
5. Store calculation history in a list
6. Display history using string formatting

Input/Output Example

=== Comprehensive Calculator ===

1. Arithmetic Operations
2. Comparison Operations
3. Logical Operations
4. String Operations
5. Collection Operations
6. View History
7. Exit

Enter choice: 1

Enter first number: 10

Enter second number: 3

Select operation (+, -, *, /, //, %, **)

: **

Result: 1000

Enter choice: 4

Enter first string: Hello

Enter second string: World

Select operation (concat, repeat): concat

Result: HelloWorld

Enter choice: 6

=== Calculation History ===

1. 10 ** 3 = 1000
2. Hello + World = HelloWorld

Enter choice: 7

Thank you for using the calculator!

Constraints

- Validate all inputs
- Handle division by zero
- Use appropriate data types and implement proper error handling

```
history = []

# Arithmetic Operations
def arithmetic_operations():
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        op = input("Select operation (+, -, *, /, //, %, **): ")
        if op == "+":
            result = num1 + num2
        elif op == "-":
            result = num1 - num2
        elif op == "*":
            result = num1 * num2
        elif op == "/":
            if num2 == 0:
                print("Error: Division by zero!")
                return
            result = num1 / num2
        elif op == "//":
            if num2 == 0:
                print("Error: Division by zero!")
                return
            result = num1 // num2
        elif op == "%":
            if num2 == 0:
                print("Error: Division by zero!")
                return
            result = num1 % num2
        elif op == "**":
            result = num1 ** num2
        else:
            print("Invalid operation!")
            return

        print("Result:", result)
        history.append(f"{num1} {op} {num2} = {result}")

    except ValueError:
        print("Invalid numeric input!")

# Comparison Operations
def comparison_operations():
    try:
```

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("\nComparison Results:")
print(f"{num1} > {num2} : {num1 > num2}")
print(f"{num1} < {num2} : {num1 < num2}")
print(f"{num1} >= {num2} : {num1 >= num2}")
print(f"{num1} <= {num2} : {num1 <= num2}")
print(f"{num1} == {num2} : {num1 == num2}")
print(f"{num1} != {num2} : {num1 != num2}")
history.append(f"Compared {num1} and {num2}")
except ValueError:
    print("Invalid numeric input!")

# Logical Operations
def logical_operations():
    try:
        age = int(input("Enter age: "))
        income = float(input("Enter income: "))

        eligible = age >= 18 and income >= 10000
        senior = age >= 60 or income >= 50000
        not_adult = not (age >= 18)

        print("\nLogical Results:")
        print("Eligible:", eligible)
        print("Senior Benefit:", senior)
        print("Not Adult:", not_adult)

        history.append(
            f"Logical Check -> Age:{age}, Income:{income}")
    except ValueError:
        print("Invalid input!")

# String Operations
def string_operations():
    str1 = input("Enter first string: ")
    str2 = input("Enter second string: ")

    op = input("Select operation (concat/repeat): ").lower()

    if op == "concat":
        result = str1 + str2

        print("Result:", result)

        history.append(
            f"{str1} + {str2} = {result}"
        )

    elif op == "repeat":
```

```
try:
    times = int(input("Repeat how many times? "))

    result = str1 * times

    print("Result:", result)

    history.append(
        f"{str1} repeated {times} times = {result}"
    )

except ValueError:
    print("Invalid repetition count!")

else:
    print("Invalid operation!")

# Collection Operations
def collection_operations():

    numbers = []

    try:
        n = int(input("How many numbers to add to the list? "))

        for i in range(n):
            value = float(input(f"Enter number {i+1}: "))
            numbers.append(value)

        print("\nList:", numbers)

        if numbers:
            print("Maximum:", max(numbers))
            print("Minimum:", min(numbers))
            print("Sum:", sum(numbers))

            history.append(
                f"List Operation on {numbers}"
            )

        except ValueError:
            print("Invalid input!")

# View History
def view_history():

    print("\n=== Calculation History ===")

    if not history:
        print("No calculations performed yet.")
    else:
```



```
        for i, item in enumerate(history, start=1):
            print(f"{i}. {item}")

while True:

    print("\n=== Comprehensive Calculator ===")
    print("1. Arithmetic Operations")
    print("2. Comparison Operations")
    print("3. Logical Operations")
    print("4. String Operations")
    print("5. Collection Operations")
    print("6. View History")
    print("7. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        arithmetic_operations()

    elif choice == "2":
        comparison_operations()

    elif choice == "3":
        logical_operations()

    elif choice == "4":
        string_operations()

    elif choice == "5":
        collection_operations()

    elif choice == "6":
        view_history()

    elif choice == "7":
        print("Thank you for using the calculator!")
        break

    else:
        print("Invalid choice! Please try again.")
```

```
=== Comprehensive Calculator ===
1. Arithmetic Operations
2. Comparison Operations
3. Logical Operations
4. String Operations
5. Collection Operations
6. View History
7. Exit
Enter choice: 1
Enter first number: 10
```

```
Enter second number: 3
Select operation (+, -, *, /, //, %, **): **
Result: 1000.0
```

```
=== Comprehensive Calculator ===
```

1. Arithmetic Operations
2. Comparison Operations
3. Logical Operations
4. String Operations
5. Collection Operations
6. View History
7. Exit

```
Enter choice: 4
Enter first string: Hello
Enter second string: World
Select operation (concat/repeat): concat
Result: HelloWorld
```

```
=== Comprehensive Calculator ===
```

1. Arithmetic Operations
2. Comparison Operations
3. Logical Operations
4. String Operations
5. Collection Operations
6. View History
7. Exit

```
Enter choice: 6
```

```
=== Calculation History ===
```

1. 10.0 ** 3.0 = 1000.0
2. Hello + World = HelloWorld

```
=== Comprehensive Calculator ===
```

1. Arithmetic Operations
2. Comparison Operations
3. Logical Operations
4. String Operations
5. Collection Operations
6. View History
7. Exit

```
Enter choice: 7
Thank you for using the calculator!
```